Python Coding in the Age of AI

CMSC 104 Section 2
May 4, 2026

Administrative Notes

Project 3 due tonight

Classwork 8 due next Monday (May 11)

Quiz 4 on Wednesday

About today's lecture

I always toss in one lecture towards the end of the semester that tries to relate the material you've learned to the "real world"

- To show you how your new skills might impact your career & lifestyle

I often make it about Juypter Notebooks - see https://jupyter.org/

- They are often used by researchers as a way of presenting research results
- You can combine real-time Python analysis with explanatory material in a single document

But I decided to go with AI this semester because it's more topical

Need an example Jupyter notebook to share

-see https://jupyter.org/try-jupyter/notebooks/index.html?path=notebooks%2FLorenz.ipynb

This lecture

All semester long you've learned how to write simple (and not-so-simple) Python programs

The purpose of this has been to get you comfortable with using the technology to solve your problems, whether those are research issues or everyday work issues

- To get you "computer literate" - meaning you understand:
  - How computers work and why they work that way
  - How to figure out the best way to solve a problem, and to know when a computer or language or program can't solve your problem

And you've learned that just in time to never have to write your own programs because some AI application can do it for you!!

“Vibe coding”

Original definition: https://x.com/karpathy/status/1886192184808149383

“There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. “

You don’t really write code. You just tell the coding system - e.g., the chatbot - what you want a program to do. And the coding system writes the code for you.

Presto

- No need to learn to code.
- No need to pay developers exorbitant salaries to write code.
- No need to understand what you’re doing; or why you’re doing it; or whether the answer is right or wrong.

You just kind of describe what you want done, and somebody else’s software does it for you.

Except it’s not quite that easy, yet.

Artificial Intelligence (AI)

“Artificial Intelligence” or “AI” is a subfield of computing that historically comes to the forefront of the mind every 10-20 years - and it’s hot right now

- The term “artificial intelligence” was coined by John McCarthy in 1956
  - He, Marvin Minsky, Claude Shannon and Nathaniel Rochester were the initial leading group
  - The “Dartmouth Summer Research Project on Artificial Intelligence”
- Artificial “neural networks” - “Perceptron” - Rosenblatt, 1957-58
  -

Important events in the history of AI

- **1964–1966** — **ELIZA**, an early natural-language chatbot, created by Joseph Weizenbaum.
- **1976** — **Mycin**, an expert system for medical diagnosis, demonstrates promise in rule-based AI.
- **1980–1987** — Widespread commercial adoption of expert systems; AI becomes a major business area.
- 1990s:
    - **1997** — IBM's **Deep Blue defeats world chess champion Garry Kasparov**, a milestone in symbolic AI.
    - Statistical learning methods (SVMs, Bayesian networks) gain prominence.
- 2000s: "Big Data" and probabilistic AI
    - **2006** — Geoffrey Hinton and others revive **deep learning** through unsupervised pre-training.
- 2010s: "Deep Learning"
    - **2011** — IBM **Watson** wins *Jeopardy!*.
    - **2012** — AlexNet wins the ImageNet competition, showing deep learning's power.
    - **2014** — Generative Adversarial Networks (GANs) introduced by Ian Goodfellow.
    - **2016** — **AlphaGo defeats Go champion Lee Sedol**, previously thought decades away.

2020s — Foundation Models & Generative AI

- **2020** — GPT-3 demonstrates large-scale transformer capabilities.
- **2022** — Breakthrough diffusion models enable realistic AI image generation.
- **2022–2024** — Rapid adoption of **large language models (LLMs)** (e.g., ChatGPT, GPT-4, Claude, Gemini).
- **2023–2024** — Emergence of **multimodal AI** and agents with reasoning and tool-use abilities.
-

So what's an "LLM?"

A **large language model** (**LLM**) is a neural network trained on a vast amount of text for natural language processing tasks, especially language generation. LLMs can generate, summarize, translate and parse text in many contexts, and are a foundational technology behind modern chatbots

(See https://en.wikipedia.org/wiki/Large_language_model)

Write software that reads everything on the Internet

- Learn what goes with what - if somebody types a question, what is the most likely answer? If someone starts typing a sentence, what is the most likely conclusion to that sentence?
- If someone tries to solve a coding problem, what is the most common solution to that problem?

It's important to differentiate between the Chatbot (front end) and the LLM (backend) that underlies it

You as a user interact with a chatbot: ChatGPT, Claude, Gemini, CoPilot, Grok,...

The chatbot relies on the LLM behind it to generate a response:

- GPT 5, Claude, LLama, Grok, Prometheus, Gemini

How accurate is the answer going to be?

“Pretty good” for common questions/tasks

- Most models these days can handle routing questions/tasks pretty well

Is your problem truly unique?

- AI can only “learn” from what has gone before - it can’t (yet) solve unique new problems very well

Is there a large corpus of training material?

- RAG - retrieval-augmented generation - tell the model to focus its answer around a corpus of material that you know to be relevant

Prompt engineering

- How clear is your question?

“Prompt Engineering”

You have to tell the LLM what you need, in sufficient clarity for it to figure out in its own knowledge base what it is you want and how to provide it

There’s often a process:

- You type a prompt
- The response is not quite exactly what you need
- So you modify your prompt to be clearer
- The response is better

Sometimes the whole process goes off the rails; the LLM “hallucinates” an answer that does not really exist

- You the user had better be able to recognize that!!

What’s an “Hallucination”

A response generated by AI that contains false or misleading information presented as fact. This term draws a loose analogy with human psychology, where a hallucination typically involves false *percepts*. However, there is a key difference: AI hallucination is associated with erroneously constructed responses (confabulation), rather than perceptual experiences

(See https://en.wikipedia.org/wiki/Hallucination_(artificial_intelligence))

How can an LLM write code?

They've read/analyzed/been trained on literally millions of Python programs

- Other languages too, but we only care about Python

The models can figure out what makes a program work:

- What the syntax is - programs written by LLMs only rarely make syntax errors

Whatever you want to do, it's likely that somebody else has done the same thing, or something very similar

- The LLM just "learns" from programs it has read, and figures out how to respond to your prompt

Coding

We can just ask an AI model to write code for us

- Just ask ChatGPT or Claude

Or we can use an AI tool custom made to write code

- Cursor.ai - developed as a plugin to VSCode (the tool many of you have been using in this class)

All of those have a “free” option that allows limited use

- That will be used for your assignment in this module

Please don’t pay subscription fees just to have access to these - wait until you really need access and get your employer or grant funder to pay!

Some examples

Next we’ll show what happens when you do some of the Python programming tasks using various AI coding tools.

Can you avoid hallucinations when coding with AI?

No. They’re caused by faulty logic in the LLM, or inadequate training, or any number of circumstances beyond your control.

You have to understand coding well enough to recognize them.

Just as you have to understand coding well enough to deal with ordinary bugs produced by the code.

How good is the code generated by AI?

It varies.

Informal surveys of professional coders indicate that it’s at about the level of a junior software developer

- It’s maybe as good as someone you just hired out of college
- But an experienced software developer can do much better, much faster.